

INF4033 – Programmation système et parallèle

Cours 4 - Les processus

Alexandre BRIÈRE



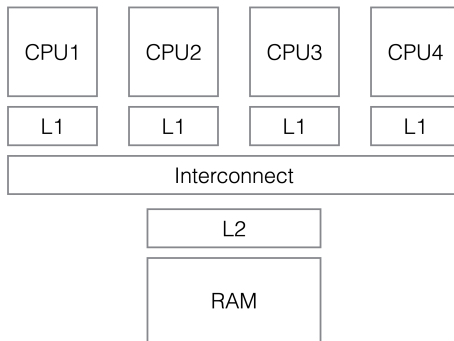
Qu'est-ce qu'un processus ?

- Un programme est une suite d'instructions interprétables par une machine
 - un programme est passif
 - il occupe juste de la place en mémoire (disque)
- Quand on l'exécute, il utilise :
 - de la mémoire (vive)
 - du temps CPU

⇒ il devient *actif* et on parle alors de *processus*

Pourquoi utiliser des processus ?

- Machines mono-processeur $\Rightarrow$ pseudo parallélisme
 - Gérer plusieurs fonctionnalités en même temps (calcul et E/S)
 - Gérer plusieurs utilisateurs en même temps
 - Répondre à des requêtes ou à des événements externes
- Machines multi-processeurs et/ou multi-cœurs $\Rightarrow$ vrai parallélisme



Composantes d'un processus

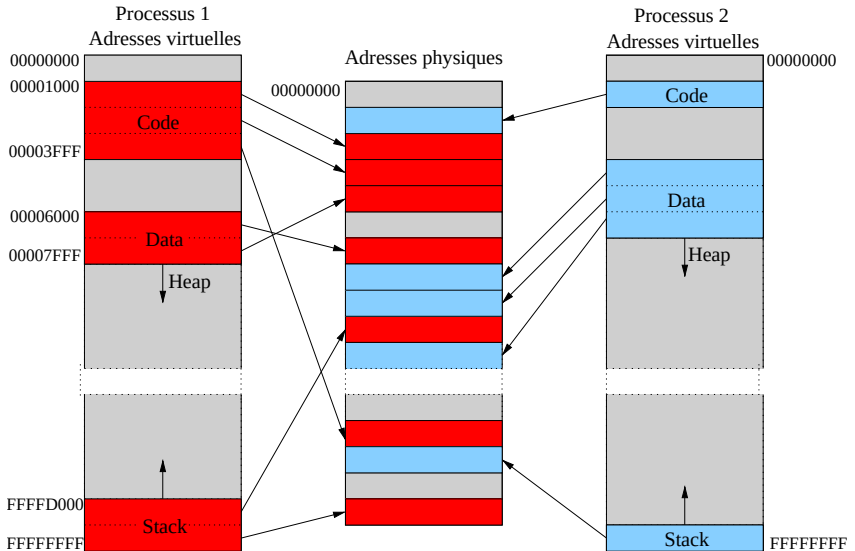
Un processus est composé de :

- un segment text $\Rightarrow$ le code source
- un segment data $\Rightarrow$ les données
- un contexte d'exécution comprenant :
 - une pile
 - l'état des registres
 - le pointeur d'instruction (Program Counter $\Rightarrow$ \$PC)

Espace d'adressage d'un processus

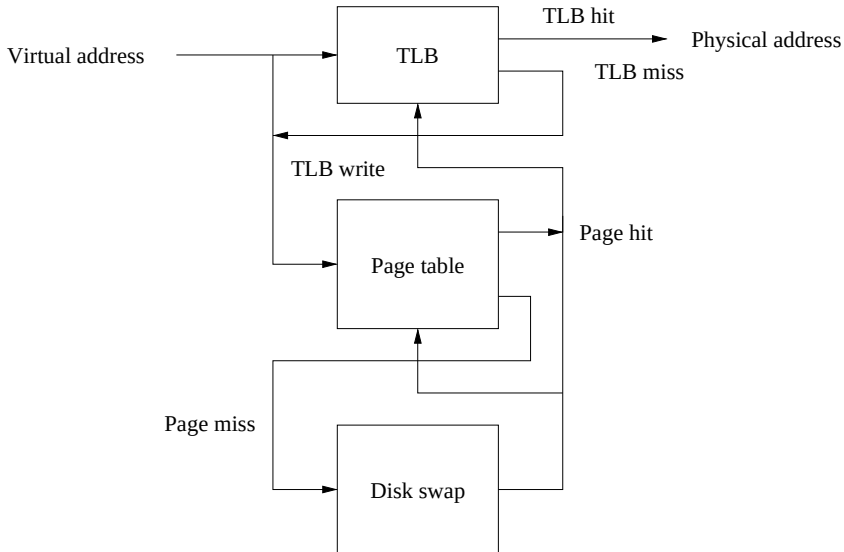
- Pile (stack)
 - variables locales
 - appels de fonctions et paramètres
 - adresse de retour de fonction
- Tas (heap)
 - allocation dynamique des variables
- Données (data et BSS)
 - constantes
 - variables statiques
- Code (text)
 - code des fonctions
 - seulement en lecture (*read-only*)

Mémoire virtuelle



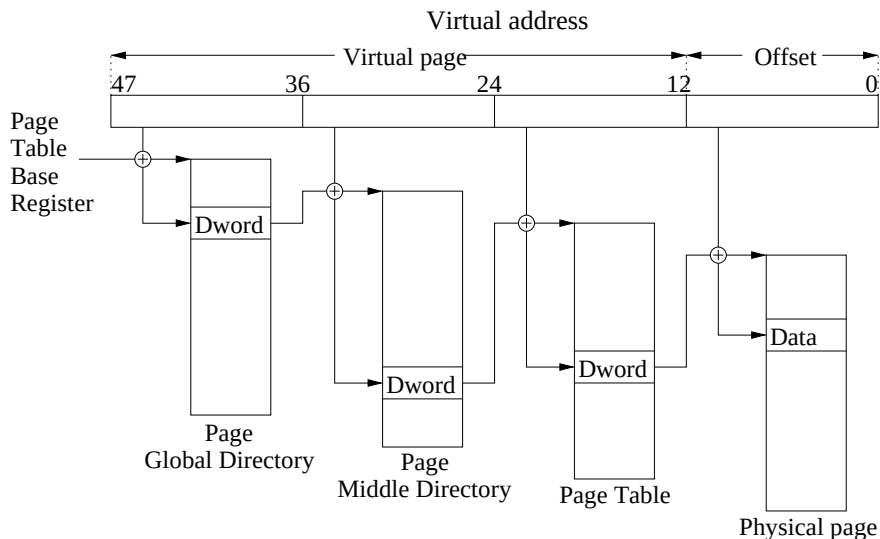
TLB (Translation Lookaside Buffer)

Traduction adresse logique → adresse physique par la TLB (1 par processus)



Mémoire paginée

Découpage de la mémoire en "pages" de taille fixe, 4 kio par exemple :)



- Visualisation instantanée
 - `ps` : les processus lancés à partir du terminal courant
 - `ps aux` : l'ensemble des processus sur la machine courante
- Visualisation en temps réel
 - `top` : affichage des processus en cours
 - `htop` : version plus moderne (interactive et visuelle)
- Visualisation généalogique
 - `pstree`

- Lancement
 - ; (en avant-plan)
 - & (en arrière-plan)
- Bascule
 - fg (vers l'avant-plan)
 - bg (vers l'arrière-plan)
- État
 - jobs
 - \$\$
 - \$?
- Envoie de signaux
 - ctrl+C (arrêt)
 - ctrl+Z (suspension)
 - kill
- Exécution conditionnelle
 - && (si OK)
 - || (si KO)

Ordonnancement des processus

- L'ordonnanceur (*scheduler*) choisit les processus à exécuter
- Gestion de la mémoire virtuelle (espace d'adressage) et du temps partagé (accès CPU) à deux niveaux :
 - sélection du prochain processus à charger en mémoire (mémoire virtuelle)
 - sélection du prochain processus (CPU) parmi ceux qui sont prêts à exécuter
- Deux classes d'ordonnanceurs :
 - non préemptifs : un processus rend le CPU quand il a terminé ou fait une E/S
 - préemptifs : l'utilisation du CPU se fait par quantum de temps
- Suivant différents critères :
 - Équité : chaque processus s'exécute à son tour
 - Temps réel : exécution des processus en fonction de priorités
- Différents algorithmes d'ordonnancement : FCFS, SJF, RR, etc.

Quelques fonctions et variables à explorer...

```
extern char **environ;  
pid_t fork(void);  
int nice(int inc);  
void _exit(int status);  
pid_t getpid(void);  
pid_t getppid(void);  
pid_t waitpid(pid_t pid, int *stat_loc, int options);  
int execl(char *path, const char *arg, ...);  
int execlp(char *file, const char *arg, ...);  
int execv(const char *path, char *const argv[]);  
int execvp(const char *file, char *const argv[]);  
...
```

- [1] **Éric RENAULT :**
Système d'exploitation (Cours à Télécom SudParis).
- [2] **Olivier CARTON :**
Architecture (Cours à l'Université Paris Cité).